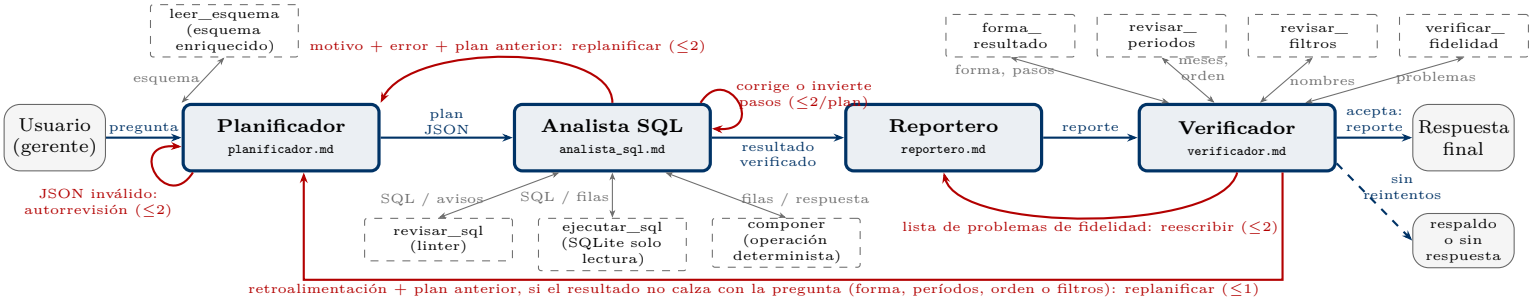
SQL4Business: Deliverable 3 — sistema de agentes

Generative Artificial Intelligence (580694), Universidad de Concepción
Aredhel Jiménez, Bryan Riquelme, Guido Salazar, Vicente Soñez

Código: github.com/aredhel-jmze/Asistente-SQL-4-Business | Video: [PENDIENTE: enlace del video]



Paradas: el Verificador acepta cuando no hay discrepancias o cuando ya no quedan reintentos. Si un plan nuevo falla con los presupuestos agotados, se entrega la respuesta anterior (*respaldo*); si el Analista agota sus correcciones sin ninguna respuesta previa, el sistema declara que no pudo responder. Azul: agentes (mismo modelo, prompt propio); gris: herramientas deterministas; rojo: ciclos y su presupuesto.

Tarea, continuidad y modelo. La misma tarea, falla y criterio de salida correcta de los entregables anteriores. Las piezas deterministas del D2 pasan a ser herramientas de los agentes y el código del D2 no cambia. Los cuatro agentes usan el mismo modelo, `Qwen/Qwen2.5-Coder-3B-Instruct` (revisión `488639f`, NF4 de 4 bits, decodificación greedy, Colab T4), con un prompt propio por agente en `prompts/`.

Agentes, decisiones y división del trabajo. El **Planificador** elige los pasos SQL y la operación de composición, que define qué calculará el Analista; si su JSON no es válido, lo revisa. El **Analista SQL** ejecuta el plan con sus herramientas y solo llama al modelo cuando una falla: el modelo decide corregir un paso, invertir dos o devolver el plan. El **Reportero** decide qué cifras citar y, si lo rechazan, reescribe. En el **Verificador**, el modelo lee solo la pregunta y declara qué pide y qué períodos nombra; una regla compara eso con sus herramientas y decide devolver el plan, devolver el reporte o aceptar. Son cuatro competencias distintas (entender, ejecutar, redactar, juzgar) y quien juzga no produce el trabajo.

Técnicas de prompting (Schulhoff et al., 2024; todos usan *role prompting* y salida estructurada). *Planificador:* (1) **few-shot**, tres ejemplos fuera de la evaluación, contra listados en vez del primero y `strftime('%Q')`; (2) **descomposición** (*plan-and-solve*): lo que pide la respuesta y los períodos antes del SQL, contra una sola consulta para preguntas de varios valores. *Analista:* (3) **autocorrección guiada por herramientas** (*self-debugging*), contra errores de fechas y sintaxis (q09, q10). *Verificador:* (4) **lista de chequeo** (*self-verification*) contrastada con herramientas, contra resultados válidos que no responden la pregunta (q03, q11, q09-p, q14-p). *Reportero:* (5) **self-refine**, contra cifras no respaldadas (q13). Cada técnica se apaga por separado.

Evaluación. 15 preguntas del D1 (10 puntuales, 5 combinadas; in-sample) y 12 paráfrasis (5/7). Mismo criterio para todos: *sql* (D1), *answer* (respuesta correcta), *e2e* (y reporte fiel).

Resultados. El sistema (v2) responde con reporte fiel 12/15 oficiales y 11/12 paráfrasis, frente a 10/15 y 10/12 del D2; en combinadas, 3/5 y 6/7 frente a 2/5 y 5/7. Frente al D2 se ganan 6 preguntas y se pierden 3 (q04, q12, q11-p); el detalle por pregunta está en el README. En la ablación, sin few-shot se pierden 2 y 1 preguntas; sin el Verificador, 1 y 1 (corrigió q11 y q08-p); sin refinamiento, un reporte (q10-p). La descomposición suma 3 combinadas externas pero cuesta una puntual oficial (q04). La autocorrección no se puede medir aquí: el Analista consultó al modelo 2 veces en 27 preguntas, porque casi no hubo errores de SQL. Con 27 preguntas, una diferencia de 1 o 2 es una tendencia. Costo: 3,6 llamadas y 33 s por pregunta (D2: 2,1 y 15 s).

Caso de falla: q12. El Planificador ordena los pasos como los nombra la pregunta (agosto antes que marzo) y el código calcula una caída de 37,14 % en vez de un alza de 59,08 %. El Verificador lo detecta (control de orden de `revisar_periodos`) y lo devuelve con el motivo exacto, pero el Planificador, con decodificación greedy y un sesgo fuerte por el orden de la pregunta, repite el mismo plan; agotado el reintento, el sistema acepta. En q04 (falta el filtro *Alimentos*) pasa lo mismo: detectar no basta si el agente que corrige ignora la retroalimentación. Trazas: `trazas/official_q12.json` y `demo/q12.json` (en `results/deliverable3/`, la segunda con el control de orden registrado).

Límites. El Verificador depende de que el modelo clasifique bien la pregunta (en q10 la lee como lista). Conjuntos chicos y sintéticos.

Desviaciones declaradas. Linter `revisar_sql` y una regla de fidelidad compartida con el D2 que no altera sus veredictos. Tras v1, el ejemplo de cambio porcentual del Reportero solo aparece si aplica, el Verificador suma `revisar_filtros`, orden y pasos descartados, y el Analista puede invertir pasos; por eso las paráfrasis ya no son estrictamente externas.

Reproducción. `notebooks/deliverable3_demo.ipynb` (Colab T4): demo (`run_agents.py`) y evaluación (`evaluate_d3.py`), con trazas en `results/`.

	Set oficial (15)			Paráfrasis (12)			
Configuración	sql	answer	e2e	sql	answer	e2e	**Llamadas**
Prompt directo (baseline D1)	7/15	8/15	–	3/12	5/12	–	1.0
Solución D2 (pipeline)	11/15	11/15	10/15	8/12	10/12	10/12	2.1
Agentes v1 (primera corrida)	11/15	11/15	9/15	9/12	11/12	9/12	3.8
Sistema de agentes v2 (D3)	11/15	12/15	12/15	9/12	11/12	11/12	3.6
sin ejemplos (few-shot)	9/15	10/15	10/15	8/12	10/12	10/12	3.4
sin descomposición explícita	10/15	13/15	13/15	6/12	8/12	8/12	3.3
sin autocorrección del Analista	11/15	12/15	12/15	9/12	11/12	11/12	3.5
sin verificación crítica	11/15	11/15	11/15	8/12	10/12	10/12	2.1
sin refinamiento del reporte	11/15	12/15	12/15	9/12	11/12	10/12	3.6